



UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA
CICLO 6

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y
SISTEMAS

“PRODUCTOR CONSUMIDOR”

INFORME 05

SISTEMAS OPERATIVOS

PRESENTA

ALEX JEANPIER CHATA CHINO
2024-119012

30 de septiembre del 2026

1. Resumen

El presente informe detalla el desarrollo y análisis de la práctica titulada "PRODUCTOR CONSUMIDOR". El objetivo principal de la experiencia fue estudiar los problemas clásicos de concurrencia mediante el uso de hilos POSIX (pthreads) y semáforos en C. A través del análisis progresivo de cuatro programas (desde una implementación con condición de carrera hasta la solución correcta), se observaron y diagnosticaron errores críticos de sincronización, incluyendo esperas lógicas defectuosas y estados de interbloqueo o *deadlock*. Finalmente, se resolvió un ejercicio práctico para consolidar la correcta inicialización y gestión de múltiples productores y consumidores sobre un búfer compartido.

2. Introducción

Imagina un escenario cotidiano en el que dos personas intentan depositar y retirar dinero de la misma cuenta bancaria exactamente en el mismo milisegundo. Si el sistema no coordina quién accede primero al saldo, el resultado final será matemáticamente incorrecto. Este es el núcleo del problema productor-consumidor, un clásico de la concurrencia en sistemas operativos. Cuando múltiples hilos de ejecución comparten recursos (como variables o búferes de memoria), es indispensable establecer mecanismos de sincronización para evitar el caos.

En esta práctica, exploraremos cómo la sincronización incorrecta puede llevar a que un hilo modifique un dato mientras otro lo lee (condición de carrera), o a situaciones donde todos los procesos se quedan esperando infinitamente porque los recursos se han bloqueado mutuamente (*deadlock*). Mediante el uso de herramientas proporcionadas por el sistema operativo, específicamente los semáforos POSIX, demostraremos cómo garantizar la exclusión mutua y la correcta coordinación entre tareas.

3. Objetivos

- Compilar y ejecutar programas multihilo utilizando el compilador gcc con la bandera `-pthread`.
- Emplear las funciones de sincronización `sem_wait` y `sem_post` y justificar técnicamente su ubicación dentro del código fuente.
- Detectar y explicar de forma argumentada condiciones de bloqueo mutuo (*deadlock*) y situaciones de espera activa o bloqueos lógicos.
- Rediseñar una solución incorrecta, analizando sus fallas, hasta obtener una versión correcta y justificable del problema.

4. Metodología

El desarrollo experimental se llevó a cabo interactuando directamente con el código fuente escrito en C (editado previamente con herramientas como gedit en Ubuntu). El procedimiento consistió en:

1. Revisión de los cuatro archivos fuente proporcionados (prodcons_01.c, prodcons_01_buggy_01.c, prodcons_01_buggy_02.c y prodcons_01_OK.c).
2. Compilación secuencial de cada programa utilizando el compilador GNU de C (gcc) enlazando la biblioteca de hilos (-lpthread).
3. Ejecución y observación detallada de la salida por consola para identificar irregularidades en la secuencia de producción y consumo de la variable compartida n.

```
1  #include <pthread.h>
2  #include <semaphore.h>
3  #include <time.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <unistd.h>
7
8  #define MAXPRODUCING 10
9  #define MAXAPPENDING 10
10 #define MAXTAKING 5
11 #define MAXCONSUMING 5
12
13 #define _wait(a) sleep(rand() % a)
14 #define out(s) \
15     printf(s); \
16     fflush(stdout)
17 #define outi(s, n) \
18     printf(s, n); \
19     fflush(stdout)
20
21 // Variables compartidas a todos los hilos
22
23 int n;
24 sem_t s, delay;
25
26 // Funciones auxiliares
27
28 void produce();
29 void append();
30 void consume();
31 void take();
32
33 // Funciones ejecutados por los hilos
34
```

```

35 void *producer(void *data)
36 {
37     while (1)
38     {
39         produce();
40         sem_wait(&s);
41         append();
42         n = n + 1;
43         outi("[P] \t \t item: %d\n", n);
44         if (n == 1)
45             sem_post(&delay);
46         sem_post(&s);
47     }
48     pthread_exit(0);
49 }
50
51 void *consumer(void *data)
52 {
53     sem_wait(&delay);
54     while (1)
55     {
56         sem_wait(&s);
57         take();
58         outi("[C] \t \t item: %d\n", n);
59         sem_post(&s);
60         consume();
61         if (n == 0)
62             sem_wait(&delay);
63     }
64     pthread_exit(0);
65 }
66
67 // Funcion principal
68

```

```
69  int main(int argc, char **argv)
70  {
71      pthread_t *consumer_pt, *producer_pt;
72      srand(time(NULL));
73      sem_init(&s, 0, 1);
74      sem_init(&delay, 0, 0);
75      consumer_pt = (pthread_t *)malloc(sizeof(pthread_t));
76      producer_pt = (pthread_t *)malloc(sizeof(pthread_t));
77      pthread_create(consumer_pt, NULL, consumer, NULL);
78      pthread_create(producer_pt, NULL, producer, NULL);
79      pthread_exit(0);
80  }
81
82
83
84  void produce()
85  {
86      out("[P] Producing\n");
87      _wait(MAXPRODUCING);
88      out("[P] Produced\n");
89  }
90
91  void append()
92  {
93      out("[P] Appending\n");
94      _wait(MAXAPPENDING);
95      out("[P] Appended\n");
96  }
97
98  void take()
99  {
100     out("[C] Taking\n");
101     _wait(MAXTAKING);
102     out("[C] Taked\n");
103 }
104
105 void consume()
106 {
107     out("[C] Consuming\n");
108     _wait(MAXCONSUMING);
109     out("[P] Consumed\n");
110 }
111
112
```

Figura 1. Código fuente de `prodcons_01.c` en el editor. Esta figura muestra el estado inicial del código base, donde se aprecian las funciones del productor y el consumidor, además de las variables compartidas y los semáforos `s` y `delay`.

```
jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src$ gcc prodcons_01.c -o prodcons1 -pthread
jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src$ ./prodcons1
```

Figura 2. Compilación del programa base mediante terminal. En esta captura se evidencia el comando de compilación exitoso utilizando `gcc` con la directiva `-pthread`, paso indispensable para habilitar el uso de hilos POSIX en Linux.

5. Desarrollo de la práctica

5.1 PROGRAMA 1: `prodcons_01.c`

Objetivo del experimento: Analizar una implementación defectuosa basada en la figura 5.9 de Stallings que presenta una condición de carrera.

Mecanismos empleados: Un semáforo `s` inicializado en 1 (que actúa como *mutex*) y un semáforo `delay` inicializado en 0 (para coordinar cuando el búfer tiene elementos).

Comportamiento observado: El programa inicia, pero la sincronización falla esporádicamente. El consumidor comprueba la variable compartida `n` después de liberar la exclusión mutua.

Análisis técnico: El problema radica en la estructura del consumidor. En este código, la instrucción `if (n == 0)` se ejecuta *después* de haber invocado `sem_post(&s)`. Al estar fuera de la zona crítica, el hilo productor puede haber modificado el valor de `n` (produciendo un nuevo elemento) en el breve lapso que transcurre entre el `sem_post(&s)` y la evaluación del `if (n == 0)`. Esto genera una condición de carrera, provocando que un aviso crucial del productor hacia el semáforo `delay` se pierda y el consumidor se quede bloqueado indefinidamente.

```

jeanpier@jeanpiernet:~/Documents/TEMP/03_producerConsumer/src$ ./prodcons1
[P] Producing
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C] item: 1
[C] Consuming
[P] Appending
[P] Consumed
[P] Appended
[P] item: 2
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C] item: 2
[C] Consuming
[P] Appending
[P] Consumed
[P] Appended
[P] item: 3
[P] Producing
[C] Taking
[C] Taked
[C] item: 3
[C] Consuming

```

Figura 3. Ejecución de `prodcons_01.c` y salida observada. La terminal muestra cómo la salida pierde consistencia debido a que el hilo consumidor lee un estado de n desfasado respecto a la realidad del sistema.

5.2 PROGRAMA 2: `prodcons_01_buggy_01.c`

- **Objetivo del experimento:** Identificar el impacto de un error semántico directo en las operaciones de semáforos.
- **Explicación del código:** El programa es casi idéntico al anterior, pero al inicio de la función `consumer`, la operación que debería detener al consumidor si no hay datos ha sido alterada.
- **Comportamiento observado y Análisis técnico:** La instrucción incorrecta es `sem_post(&delay);` // un tipo de error. Al usar `sem_post` en lugar de `sem_wait`, el consumidor incrementa el valor del semáforo en lugar de esperar a que el productor genere el primer ítem. Como consecuencia, si el planificador de procesos decide ejecutar primero al consumidor, este no se detendrá: entrará al ciclo, adquirirá el semáforo `s` y

comenzará a "retirar" elementos fantasma, provocando que la variable n adopte valores negativos inconsistentes.

```
jeanpier@jeanpiernet:~/Documents/TEMP/03_producutorConsumidor/src$ ./prodcons2
[C] Taking
[P] Producing
[C] Taked
[C]          item: 0
[C] Consuming
[P] Produced
[P] Appending
[P] Appended
[P]          item: 0
[P] Producing
[P] Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: 0
[C] Consuming
[P] Appending
[P] Consumed
[P] Appended
[P]          item: 0
[P] Producing
[C] Taking
[C] Taked
[C]          item: 0
[C] Consuming
[P] Produced
[P] Appending
[P] Consumed
[P] Appended
[P]          item: 0
[P] Producing
[C] Taking
[C] Taked
```

Figura 4. Ejecución de `prodcons_01_buggy_01.c`. En esta captura se observa al consumidor procesando ítems antes de que el productor haya tenido tiempo de incorporarlos, evidenciando el fallo en la contención inicial.

5.3 PROGRAMA 3: `prodcons_01_buggy_02.c`

- **Objetivo del experimento:** Visualizar la generación de un interbloqueo (*deadlock*) ocasionado por el orden en que se adquieren los candados.
- **Explicación del código:** En esta versión, dentro del ciclo del consumidor, el orden original fue invertido: primero se ejecuta `sem_wait(&s)`; y posteriormente `sem_wait(&delay)`;
- **Comportamiento observado y Análisis técnico:** El programa se "congela". Este es un

ejemplo clásico de *deadlock*. El flujo fatal es el siguiente: el consumidor entra, adquiere el semáforo *s* (el cerrojo global). Como el búfer está vacío, al llegar a la siguiente línea, se bloquea en *sem_wait(&delay)* esperando que el productor genere algo. Sin embargo, cuando el productor intenta generar algo, se queda bloqueado indefinidamente en su propio *sem_wait(&s)* porque el consumidor tiene el control del candado *s* y no lo soltará hasta pasar el *delay*. Ninguno de los dos hilos puede avanzar.

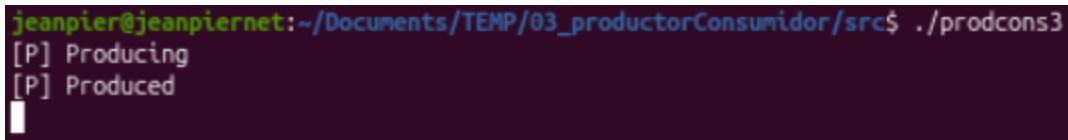
A terminal window with a dark background. The prompt is 'jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src\$'. The command './prodcons3' has been executed. The output shows '[P] Producing' followed by '[P] Produced' on the next line. The cursor is on a new line, and the terminal appears to be frozen, indicating a deadlock state where both processes are waiting for resources that will never be available.

Figura 5. Ejecución de *prodcons_01_buggy_02.c*. La captura muestra la terminal estática (congelada), demostrando gráficamente cómo ambos procesos han entrado en una condición de bloqueo mutuo de la cual no pueden recuperarse.

5.4 PROGRAMA 4: *prodcons_01_OK.c*

- **Objetivo del experimento:** Analizar la solución correcta propuesta en la figura 5.10 de Stallings, que utiliza una variable auxiliar.
- **Explicación de la solución:** Dentro de la función *consumer*, se declara una variable local *int m*;. Antes de abandonar la sección crítica protegida por *s*, el consumidor guarda el valor actual de *n* ejecutando *m = n*;
- **Análisis técnico:** Esta sencilla modificación soluciona la condición de carrera descubierta en el primer programa. Al almacenar el estado de la variable global *n* en la memoria local del hilo (*m*) antes de ejecutar *sem_post(&s)*, garantizamos que la evaluación posterior *if (m == 0)* se basará en el valor exacto que existía al momento de consumir el recurso, ignorando si el productor ha modificado *n* entre tanto. El acceso concurrente está finalmente securizado.

```

jeanpier@jeanpiernet:~/Documents/TEMP/03_producerConsumidor/src$ ./prodcons4
[P] Producing
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taked
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending
[P] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taked
[C] item: 1
[C] Consuming
[P] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1

```

Figura 6. Ejecución exitosa de *prodcons_01_OK.c*. La ejecución fluye de manera sostenida, demostrando que el uso de la variable de estado local protege al consumidor de lecturas corrompidas, sincronizando la producción y el consumo perfectamente.

5.5 Análisis comparativo de los 4 programas

Programa	Mecanismo Modificado	Problema Identificado	Comportamiento Observado
prodcons_01.c	Uso del semáforo s sin proteger la lectura de n.	Condición de carrera.	Sincronización inestable, pérdida de señales <i>wake-up</i> .
prodcons_01_buggy_01.c	sem_post(&delay) en vez de sem_wait.	Error semántico de bloqueo inicial.	El consumidor procesa datos inexistentes (bajo cero).
prodcons_01_buggy_02.c	Inversión: wait(&s) antes de wait(&delay).	Interbloqueo / <i>Deadlock</i> .	El programa se congela. Bloqueo circular permanente.
prodcons_01_OK.c	Variable local m captura el valor de n.	Corrección de la condición de carrera.	Flujo continuo, exclusión mutua respetada estrictamente.

6. Análisis de resultados

Nota: Las respuestas resumidas se encuentran en el archivo markdown adjunto para GitHub. A continuación, el análisis riguroso para el informe.

1. **Sobre prodcons_01.c y la condición de carrera:** La sincronización falla porque el consumidor evalúa el estado del sistema ($n == 0$) fuera de la zona crítica. Al liberar el cerrojo s antes de evaluar, abrimos una ventana de tiempo en la que el hilo productor puede tomar control del procesador, alterar la variable n e intentar señalar a $delay$. Como el consumidor aún no ha evaluado el if , ignorará esta señal, y al reanudar su ejecución se bloqueará en un $delay$ que ya fue emitido y desperdiciado.
2. **Sobre prodcons_01_buggy_01.c y la instrucción incorrecta:** La línea incorrecta es `sem_post(&delay)`; al inicio de la función del consumidor. Esta instrucción otorga permiso inmediato para avanzar. Si el consumidor, por planificación del núcleo, inicia su ejecución antes que el productor, el consumidor retirará elementos que no existen, comprometiendo la integridad de la variable compartida e invalidando el propósito de la sincronización.
3. **Sobre prodcons_01_buggy_02.c y el interbloqueo:** El bloqueo cruzado o *deadlock* es inevitable porque el consumidor toma el control total del acceso exclusivo (esperando en s) y se detiene a la espera de un evento (esperando en $delay$) que *sólo* puede ser generado por el productor. Como el productor necesita desesperadamente la llave s para generar dicho evento, ambos quedan en una paralización infinita.
4. **Sobre prodcons_01_OK.c y la variable auxiliar:** La incorporación de $m = n$; dentro de las fronteras de `sem_wait(&s)` y `sem_post(&s)` soluciona el problema creando una "fotografía" segura del estado. Al evaluar m en lugar de la variable viva n , la decisión del consumidor sobre si debe dormir o no, es completamente inmune a las modificaciones concurrentes que el productor pueda estar realizando en paralelo.

7. Ejercicio final

La guía propone completar el código de un sistema productor-consumidor utilizando un búfer circular de tamaño 5 ($BUFF_SIZE = 5$), con 3 productores y 3 consumidores. A continuación se presenta el código funcional con las correcciones sintácticas necesarias (se eliminaron los caracteres defectuosos que generaban errores como `&shared.empty`; por `&shared.empty`):

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6  #include <stdint.h>
7
8  #define BUFF_SIZE 5      /* total number of slots */
9  #define NP 3            /* total number of producers */
10 #define NC 3            /* total number of consumers */
11 #define NITERS 4        /* number of items produced/consumed */
12
13 You, 6 seconds ago | author (You)
14 typedef struct {
15     int buf[BUFF_SIZE];  /* shared var */
16     int in;               /* buf[in%BUFF_SIZE] is the first empty slot */
17     int out;              /* buf[out%BUFF_SIZE] is the first full slot */
18     sem_t full;           /* keep track of the number of full spots */
19     sem_t empty;          /* keep track of the number of empty spots */
20     sem_t mutex;          /* enforce mutual exclusion to shared data */
21 } sbuf_t;
22
23 sbuf_t shared;
24
25 void *Producer(void *arg)
26 {
27     int i, item, index;
28     index = (int)(intptr_t)arg; /* Casting corregido para evitar warnings */
29
30     for (i=0; i < NITERS; i++) {
31         item = i;
32         /* Si no hay espacios vacios, esperar */
33         sem_wait(&shared.empty);
34         /* Si otro hilo usa el buffer, esperar (exclusion mutua) */
35         sem_wait(&shared.mutex);
36
37         shared.buf[shared.in] = item;
38         shared.in = (shared.in+1)%BUFF_SIZE;
39         printf("[P%d] Producing %d ...\n", index, item);
40         fflush(stdout);
41
42         /* Liberar el buffer */
43         sem_post(&shared.mutex);

```

```

43     /* Incrementar contador de espacios llenos */
44     sem_post(&shared.full);
45
46     if (i % 2 == 1) sleep(1);
47 }
48 return NULL;
49 }
50
51 void *Consumer(void *arg)
52 {
53     int i, item, index;
54     index = (int)(intptr_t)arg;
55
56     /* El numero total a consumir se divide entre los consumidores */
57     for (i=0; i < (NP * NITERS)/NC; i++) {
58         /* Si no hay elementos llenos, esperar */
59         sem_wait(&shared.full);
60         /* Bloquear acceso a los datos compartidos */
61         sem_wait(&shared.mutex);
62
63         item = shared.buf[shared.out];
64         shared.out = (shared.out+1)%BUFF_SIZE;
65         printf("-----> [C%d] consumed %d\n", index, item);
66         fflush(stdout);
67
68         /* Liberar el buffer */
69         sem_post(&shared.mutex);
70         /* Indicar que ahora hay un nuevo espacio vacio */
71         sem_post(&shared.empty);
72
73         if (i % 2 == 1) sleep(1);
74     }
75     return NULL;
76 }
77

```

```

78  int main()
79  {
80      pthread_t idP[NP], idC[NC];
81      int index;
82
83      sem_init(&shared.full, 0, 0);
84      sem_init(&shared.empty, 0, BUFF_SIZE);
85
86      /* Inicializar el candado mutex a 1 (disponible) */
87      sem_init(&shared.mutex, 0, 1);
88
89      for (index = 0; index < NP; index++)
90      {
91          /* Crear a los productores */
92          pthread_create(&idP[index], NULL, Producer, (void*)(intptr_t)index);
93      }
94
95      /* Crear NC consumidores */
96      for (index = 0; index < NC; index++)
97      {
98          pthread_create(&idC[index], NULL, Consumer, (void*)(intptr_t)index);
99      }
100
101      /* Esperar finalizacion o salir */
102      pthread_exit(NULL);
103  }

```

Análisis: Para que el ejercicio fuera funcional, en el método principal *main()* se implementó la inicialización del semáforo binario *mutex* con un valor de 1, esencial para asegurar la exclusión mutua sobre el arreglo *buf*. Posteriormente se implementó el bucle para la instanciación de los NC consumidores utilizando *pthread_create*. En la función *Consumer*, la lógica actúa como espejo de *Producer*: en lugar de esperar espacios vacíos (*empty*), espera ítems generados (*full*). Una vez finalizado el consumo, libera el candado *mutex* e incrementa el contador de casillas libres mediante *sem_post(&shared.empty)*. El proceso de compilación requirió enlazar tanto las bibliotecas *-lpthread* como *-lrt*.

```

jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src$ gcc prodcons_e3.c -o prodcons_e3 -lpthread -lrt
jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src$ ./prodcons_e3
[P0] Producing 0 ...
[P0] Producing 1 ...
[P1] Producing 0 ...
[P1] Producing 1 ...
[P2] Producing 0 ...
-----> [C0] consumed 0
-----> [C2] consumed 1
-----> [C2] consumed 0
-----> [C1] consumed 1
-----> [C0] consumed 0
[P2] Producing 1 ...
-----> [C1] consumed 1
[P1] Producing 2 ...
[P1] Producing 3 ...
[P0] Producing 2 ...
[P0] Producing 3 ...
-----> [C2] consumed 2
-----> [C0] consumed 3
-----> [C0] consumed 2
-----> [C2] consumed 3
[P2] Producing 2 ...
[P2] Producing 3 ...
-----> [C1] consumed 2
-----> [C1] consumed 3
jeanpier@jeanpiernet:~/Documents/TEMP/03_productorConsumidor/src$

```

Figura 7. Ejecución de *prodcon.c* (Ejercicio 3). La captura evidencia una ejecución armónica en consola donde se identifican correctamente las etiquetas [P0], [P1], [P2] en producción y [C0], [C1], [C2] durante el consumo de datos compartidos.

8. Conclusiones

1. La implementación de procesos concurrentes exige rigor en el diseño de las fronteras críticas. Como se observó en el primer programa, un solo condicional mal ubicado altera drásticamente la capacidad de los hilos para responder a señales en tiempo real.
2. El orden de las solicitudes de bloqueo es tan importante como las propias operaciones de bloqueo. Solicitar un cerrojo de recurso sin garantizar antes que existen los datos a procesar, conduce invariablemente a escenarios catastróficos de *deadlock*.
3. El uso de semáforos como contadores asíncronos (full y empty) combinados con semáforos binarios (mutex) representa el patrón de diseño más estable para aislar zonas de memoria, tal como quedó demostrado en el funcionamiento exitoso del Ejercicio 3.

9. Bibliografía

Stallings, W. (2018). *Sistemas Operativos: Aspectos internos y principios de diseño* (9.a ed.). Pearson Educación.

Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3.a ed.). O'Reilly Media.